Python 常用第三方库速查手册（分类 + 示例）

> 面向"**想用 Python 做点真东西**"的入门到进阶场景：算法、爬虫、网络与安全、数据、AI、Web、办公自动化、图像音视频、数据库、工程化。
> **怎么读**：先看「一、安装与工程基础」把环境搞对，再按需跳到感兴趣的分类。
> 每个库都给了：**定位 → 安装 → 最小可运行示例 → 备注（坑在哪）**。
> 版本提示：本文写于 2026 年，库的 API 会变，遇到报错先看官方文档的版本说明。

总览思维导图

> 下面用 Mermaid 画了整体地图。在 **VS Code** 里装 `Markdown Preview Mermaid Support` 插件后就能直接看图形；GitHub、Obsidian、Typora 也都原生支持。

【思维导图】以下为 Mermaid 源码，图形请在 Markdown 版查看

```
mindmap
  root((Python 第三方库))
    一 安装与工程
      pip
      venv 虚拟环境
      镜像源
      requirements
    二 数据与科学计算
      numpy
      pandas
      matplotlib
      scipy
      sympy
      polars
    三 算法与数据结构
      标准库神器
      collections
      heapq
      bisect
      itertools
      lru_cache
      第三方
      networkx
      sortedcontainers
      more-itertools
      z3-solver
    四 爬虫与采集
      requests
      beautifulsoup4
      lxml
      playwright
      selenium
      scrapy
    五 网络与安全
```

```
socket 基础
scapy 流量
python-nmap 扫描
cryptography 加密
bcrypt 密码
mitmproxy 抓包
防守 yara bandit pip-audit
六 机器学习与 AI
scikit-learn
xgboost
pytorch
transformers
openai
jieba 中文
七 Web 后端
fastapi
flask
django
sqlalchemy
八 办公与自动化
openpyxl Excel
python-docx Word
pdfplumber PDF
pyautogui 桌面
schedule 定时
九 图像音视频
Pillow
OpenCV
pytesseract OCR
whisper 语音
moviepy 视频
十 数据库与缓存
sqlite3
pymysql
```

```
pymongo
redis
十一 工程化
pytest
ruff
mypy
loguru
rich
typer
十二 选型与路线
按任务查库
6 条学习路线
安装命令合集
```

三条主线流程图

① 爬虫工作流

【思维导图】以下为 Mermaid 源码，图形请在 Markdown 版查看

```
flowchart LR
 A[确定目标站点] --> B{有官方 API 吗}
 B -- 有 --> C[直接调 API]
 B -- 没有 --> D[看 robots.txt 与条款]
 D --> E{页面是 JS 渲染吗}
 E -- 否 --> F[requests 抓 HTML]
 E -- 是 --> G[playwright 开浏览器]
 F --> H[bs4 或 lxml 解析]
 G --> H
 H --> I[清洗成 dict 或 DataFrame]
 I --> J[存 CSV / SQLite / JSON]
 J --> K[限速 + 重试 + 日志]
 K --> L[分析出图或生成报告]
```

② 机器学习流程

【思维导图】以下为 Mermaid 源码，图形请在 Markdown 版查看

```
flowchart LR
 A[收集数据] --> B[pandas 清洗]
 B --> C[特征工程]
 C --> D[train_test_split 切分]
 D --> E[选择模型]
 E --> F[fit 训练]
 F --> G[predict 预测]
 G --> H[评估 精度 召回 F1]
 H --> I{够好吗}
 I -- 不够 --> C
 I -- 够了 --> J[保存模型 joblib]
 J --> K[部署成 API fastapi]
```

③ 网络安全学习路线（仅授权 / 靶场）

【思维导图】以下为 Mermaid 源码，图形请在 Markdown 版查看

```
flowchart TD
 A[计算机网络 + Linux + HTTP] --> B[编程 socket 与 requests]
 B --> C[手工测试 Burp ZAP]
 C --> D[靶场刷题 PortSwigger HTB]
 D --> E{选方向}
 E --> F[Web 安全]
 E --> G[渗透测试]
 E --> H[二进制逆向 pwntools z3]
 E --> I[流量分析 scapy pyshark]
 E --> J[应急响应与取证]
 E --> K[防守 yara SIEM]
```

目录

- ● **全库用途总表（先看这一页）**
- 一、安装与工程基础（pip / venv / 镜像 / requirements）
- 二、数据处理与科学计算（numpy、pandas、matplotlib…）
- 三、算法与数据结构（标准库神器 + sortedcontainers、networkx…）
- 四、爬虫与网络采集（requests、bs4、scrapy、playwright…）
- 五、网络与安全攻防（scapy、pwntools、cryptography…）[!] 含合规声明
- 六、机器学习与 AI（scikit-learn、pytorch、transformers…）
- 七、Web 后端与接口（flask、fastapi、sqlalchemy…）
- 八、办公与自动化（openpyxl、python-docx、pyautogui…）
- 九、图像 / 音视频 / OCR（Pillow、OpenCV、moviepy…）

- 十、数据库与缓存（sqlite3、pymysql、redis…）
- 十一、测试、调试与工程化（pytest、ruff、loguru、rich…）
- 十二、快速选型表 + 学习路线

● 全库用途总表（先看这一页）

> 「**用途**」= 它能帮你解决什么问题；「**典型场景**」= 什么时候你会想起它。
> 装了之后不会用的，翻回后面看每个库的完整示例。

数据处理与科学计算

库	用途	典型场景
numpy	多维数组 + 向量化运算	矩阵计算、图像像素、统计；其他库的底座
pandas	表格的读取 / 清洗 / 筛选 / 分组 / 聚合	Excel、CSV 分析，爬虫数据整理，生成报表
polars	和 pandas 同用途，但更快更省内存	百万行以上的大表
matplotlib	画静态图	报告、论文插图
seaborn	统计图美化（基于 matplotlib）	一行画出分布图、箱线图
scipy	科学计算：优化、积分、插值、信号、统计检验	数值求解、信号处理
sympy	**符号**数学：解方程、求导、积分、化简	要推导公式而不是算数值
statsmodels	统计建模：回归、假设检验、时间序列	学术统计、需要 p 值
pyarrow	列式存储与格式转换	parquet 文件、加速 pandas

算法与数据结构

库	用途	典型场景
标准库 collections / heapq / bisect / itertools / functools	队列、计数、堆、二分、排列组合、记忆化	算法题主力，**不用装**
sortedcontainers	一直保持有序的列表 / 字典	频繁"插入 + 查找"的场景
more-itertools	itertools 的加强版	滑动窗口、分块、去重
networkx	图论：建图、最短路径、连通分量、拓扑排序	关系网络、依赖分析、路径规划
python-constraint	约束满足求解	数独、排班、选课
pulp / ortools	线性规划 / 运筹优化	资源分配、排班、路径优化
z3-solver	约束求解器	解方程组、逆向约束、CTF

爬虫与网络采集

库	用途	典型场景
requests	发 HTTP 请求	调 API、抓静态页、下载文件
httpx	异步 HTTP 请求	高并发抓取

库	用途	典型场景
beautifulsoup4	HTML 解析（容错好、好上手）	提取标题 / 价格 / 链接
lxml	高速 HTML / XML 解析	大页面、给 bs4 当解析器
parsel	XPath / CSS 提取	Scrapy 内、需要精确定位
playwright	真浏览器自动化	JS 渲染页面、截图、自动化测试
selenium	真浏览器自动化（老牌）	老项目、特定驱动
scrapy	爬虫框架	大规模、工程化抓取
fake-useragent	随机 User-Agent	降低被拦概率（合规前提下）
tqdm	进度条	长循环、下载进度
tenacity	自动重试	网络抖动、限流重试
pyquery	jQuery 风格解析 HTML	习惯 jQuery 的人
robobrowser	简化表单提交	老式登录页

网络与安全攻防 [!]（仅限自有设备 / 书面授权 / 靶场）

库	用途	典型场景
socket（标准库）	底层 TCP / UDP 编程	理解网络原理、写简单服务端
python-nmap	调用 nmap 做端口与服务探测	资产清点（**只扫自己**）
scapy	构造 / 解析 / 嗅探数据包	协议学习、流量分析
dnspython	DNS 查询与解析	域名解析排查
python-whois	域名注册信息	域名资产梳理
pyshark	解析抓包文件	流量取证
psutil	系统与网络状态	进程 / 端口监控
hashlib / hmac / secrets（标准库）	摘要、签名、安全随机	文件校验、Token 生成
cryptography	现代加解密、证书	AES / RSA、HTTPS
pycryptodome	传统加密接口	老项目、特定算法
bcrypt / argon2-cffi	**密码哈希**	用户密码入库（必须用这个）
pyjwt	JWT 签发与校验	登录态
mitmproxy	中间人代理	调试手机 / 浏览器流量
faker	生成测试数据	测试、演示（**别用真实数据**）
bandit	扫描**自己代码**的安全问题	CI 里自动检查
pip-audit	扫描依赖库的已知漏洞	上线前检查
yara-python	恶意样本特征匹配	**防守**：写规则扫文件
capstone / unicorn / pyelftools / pefile / angr	反汇编、模拟执行、解析 ELF / PE、符号执行	逆向分析、CTF
ssdeep / py-tlsh	模糊哈希	找相似样本
geoip2	IP → 地理位置	日志分析

库	用途	典型场景
elasticsearch	日志检索	SIEM / 安全分析平台

机器学习与 AI

库	用途	典型场景
scikit-learn	传统机器学习全流程	表格数据做分类 / 回归 / 聚类
xgboost / lightgbm / catboost	梯度提升树	结构化数据竞赛、Kaggle
pytorch	深度学习框架	图像 / 文本模型训练与微调
torchvision	视觉数据集 + 预训练模型	图像任务
transformers	加载 BERT / GPT 等预训练模型	文本分类、摘要、翻译、微调
datasets	一行加载公开数据集	训练数据准备
sentence-transformers	句向量、语义相似度	检索、去重、聚类
openai	调用大模型 API	聊天、写作、Agent、RAG
tiktoken	分词与 token 计数	估算 API 费用
accelerate	多卡 / 混合精度训练	训练加速
peft	LoRA 等参数高效微调	低成本微调大模型
langchain / llama-index	编排 RAG 与 Agent	知识库问答、工具调用
jieba	中文分词	词频统计、关键词、情感分析
snownlp	中文情感分析	评论正负面判断
wordcloud	词云图	可视化报告
pypinyin	汉字转拼音	排序、检索、注音
hanlp / ltp	专业中文 NLP	分词、词性、实体识别

Web 后端与接口

库	用途	典型场景
flask	轻量 Web 框架	小网站、小接口、教学
fastapi	高性能 API + **自动生成文档**	给前端 / App / AI 提供接口
uvicorn	ASGI 服务器	跑 FastAPI
gunicorn	WSGI 服务器	Linux 生产部署
pydantic	数据校验与配置	请求体 / 配置项校验
sqlalchemy	ORM	用 Python 类操作数据库
django	全家桶框架	中大型站点（自带后台、用户系统）
starlette	轻量 ASGI 框架	FastAPI 的底层
jinja2	模板引擎	渲染 HTML 页面
gradio / streamlit	几行代码做 AI 演示界面	快速分享 Demo、内部工具

办公与自动化

库	用途	典型场景
openpyxl	读写 xlsx	生成带样式的报表

库	用途	典型场景
xlrd	读老式 .xls	兼容旧文件
python-docx	读写 Word	自动生成报告 / 合同
python-pptx	读写 PPT	自动生成演示文稿
pypdf	PDF 拆分 / 合并 / 加密	批量处理 PDF
pdfplumber	提取 PDF 文字与表格	发票、报表解析
pymupdf	PDF 转图片、快速提取	扫描件、批量转换
reportlab	生成 PDF	报告、发票、证书
weasyprint	HTML → PDF	用网页排版做 PDF
pyautogui	桌面自动化	重复点击、填表
pywinauto	Windows 应用自动化	更稳定的桌面自动化
schedule	定时任务（进程内）	每天自动跑一次
apscheduler	专业调度器	cron 表达式、持久化任务
python-crontab	管理 Linux crontab	服务器定时
pyinstaller	打包成 exe	给不会 Python 的人用

图像 / 音视频 / OCR

库	用途	典型场景
pillow	图像基础处理	批量改尺寸、加水印、转格式
opencv-python	计算机视觉	人脸检测、目标跟踪、视频处理
pytesseract	OCR（Tesseract 封装）	简单图片文字识别
easyocr / paddleocr / cnocr	深度学习 OCR	中文票据、截图识别
moviepy	视频剪辑	裁剪、拼接、加字幕
ffmpeg-python	组装 ffmpeg 命令	转码、抽帧、压缩
pydub	音频切片与格式转换	剪铃声、合并音频
mutagen	读写音频元数据	批量改 ID3 标签
librosa	音频特征提取	语音 / 音乐分析
whisper	语音转文字	字幕、会议记录
qrcode	生成二维码	链接、名片、支付码演示
pyzbar	识别二维码 / 条形码	解码图片

数据库与缓存

库	用途	典型场景
sqlite3（标准库）	单文件数据库	本地存储、爬虫入库
pymysql	连接 MySQL	传统 Web 应用
psycopg	连接 PostgreSQL	对一致性要求高的系统
pymongo	连接 MongoDB	字段不固定的文档数据
redis	内存缓存 / 队列 / 限流	缓存、计数、任务队列

库	用途	典型场景
sqlalchemy	ORM	不写 SQL 操作数据库
peewee / tortoise-orm	轻量 ORM	小项目、异步项目
alembic	数据库迁移	改表结构不丢数据
duckdb	单机分析型数据库	直接用 SQL 查 CSV / Parquet

测试 / 调试 / 工程化

库	用途	典型场景
pytest	单元测试	改代码不慌
pytest-cov	测试覆盖率	看测试够不够全
ruff	代码检查 + 格式化	一个工具顶 flake8 + isort + black
black	自动格式化	统一团队风格
mypy	静态类型检查	提前抓出类型错误
isort	import 排序	让文件整洁
flake8	风格检查	老项目
pre-commit	提交前自动检查	防低级错误进仓库
loguru	日志	落盘、分级、自动带堆栈
icecream	调试打印	自动显示变量名和值
tqdm	进度条	长任务可视
rich	终端美化、表格、进度	CLI 输出好看
click / typer	命令行框架	写专业的 CLI 工具
python-dotenv	读取 .env	管理 API Key
pyyaml	读写 YAML	配置文件
orjson	更快的 JSON	大数据量序列化
ipython	交互式解释器	边写边试

怎么用好这张表

1. **不要全装。**先想清楚要做什么 → 从这张表里挑 1~2 个 → 装了就用起来。
2. **一个方向先精一个。**爬虫先 `requests` + `beautifulsoup4`，分析先 `pandas` + `matplotlib`，别一上来开五个坑。
3. **能标准库就别装库。**`collections` / `itertools` / `sqlite3` / `json` / `pathlib` / `argparse` 已经能干很多事，而且零依赖。
4. **装完先跑一遍文档里的最小示例**，确认环境没问题再写自己的代码。

一、安装与工程基础

1.1 pip —— 包管理器

```
pip install requests # 安装
pip install requests==2.32.3 # 装指定版本
pip install "requests>=2.30" # 装某个范围
pip install -U requests # 升级
pip uninstall requests # 卸载
pip list # 看装了什么
pip show requests # 看某个包的详情
```

1.2 虚拟环境 —— 每个项目一个，非常重要

```
python -m venv .venv # 创建
.venv\Scripts\activate # Windows 激活
source .venv/bin/activate # macOS / Linux
deactivate # 退出
```

> [!] **为什么要用**：不同项目依赖的版本可能冲突（A 项目要 numpy 1.x，B 项目要 2.x）。
> 全局装会互相打架，虚拟环境让每个项目各玩各的。
> VS Code 里按 Ctrl+Shift+P → Python: Select Interpreter → 选 .venv。

1.3 国内镜像源（装包提速）

```
pip install requests -i https://pypi.tuna.tsinghua.edu.cn/simple
# 永久设置
pip config set global.index-url https://pypi.tuna.tsinghua.edu.cn/simple
```

常用镜像：清华 TUNA、阿里云（https://mirrors.aliyun.com/pypi/simple/）、腾讯云、中科大。

1.4 requirements.txt —— 记录依赖

```
pip freeze > requirements.txt # 导出当前环境
pip install -r requirements.txt # 照单安装
```

1.5 一分钟判断"该用哪个库"

想干什么	直接上
发 HTTP 请求 / 调接口	requests / httpx
解析网页	beautifulsoup4 + lxml
大规模爬虫	scrapy
要执行 JS 的网页	playwright / selenium
表格数据分析	pandas
数值矩阵计算	numpy
画图	matplotlib
机器学习	scikit-learn
深度学习	pytorch
Excel 读写	openpyxl / pandas
Word 读写	python-docx
PDF 提取文字	pdfplumber
图像处理	Pillow / opencv-python
网页自动化	playwright
桌面自动化	pyautogui

想干什么	直接上
写 Web 接口	`fastapi`
写网站	`django` / `flask`
写命令行工具	`click` / `typer` + `rich`
加密 / 哈希	`cryptography` / 内置 `hashlib`
写测试	`pytest`
打日志	`loguru`

二、数据处理与科学计算

numpy —— 数值计算的地基

定位：多维数组 + 向量化运算。几乎所有数据/科学库都建在它上面。

用途：批量数值计算（**不写循环**做整体运算）：矩阵运算、统计、图像像素处理；它还是 pandas / OpenCV / PyTorch 等一大票库的底座

用途：把项目用到的库和精确版本记成一张清单；换电脑、交给同学、部署服务器时一条 `pip install -r` 装齐

用途：给每个项目一套**独立的库目录**，避免 A 项目要 numpy 1.x、B 项目要 2.x 时互相打架；不要了直接删掉整个文件夹

用途：安装 / 升级 / 卸载第三方库，锁定版本，导出依赖清单 —— **用别人造好的轮子，不用从零写**

安装：`pip install numpy`

```
import numpy as np

a = np.array([1, 2, 3, 4])
print(a * 2) # [2 4 6 8] 整体运算，不用写循环
print(a.mean(), a.sum(), a.max())

m = np.arange(12).reshape(3, 4) # 3 行 4 列
print(m.shape, m.dtype) # (3, 4) int64
print(m[1, 2]) # 第 2 行第 3 列
print(m[:, 1]) # 第 2 列
print(m @ m.T) # 矩阵乘法
print(np.linspace(0, 1, 5)) # [0. 0.25 0.5 0.75 1.]
```

备注

- [√] **别用 for 循环遍历 numpy 数组**——它真正快在"整体运算"（向量化）
- [!] `a * b` 是**逐元素相乘**，矩阵乘法要写 `a @ b`
- [!] 切片返回的是**视图**（改子数组会改原数组），要副本用 `.copy()`
- [!] 整数数组除以整数会得到浮点（真除法），整除用 `//`

pandas —— 表格数据分析

定位：像"Python 里的 Excel"，处理带列名的二维表。

用途：把 CSV / Excel / SQL 读成带列名的表，做筛选、分组、聚合、合并、透视 —— **数据分析的日常主力**

安装：`pip install pandas`

```
import pandas as pd

df = pd.DataFrame({
 "姓名": ["张三", "李四", "王五"],
 "分数": [686, 655, 700],
 "城市": ["杭州", "武汉", "北京"],
})

print(df.head()) # 前 5 行
print(df["分数"].mean()) # 平均分
print(df[df["分数"] >= 686]) # 按条件筛选
print(df.sort_values("分数", ascending=False))
df["是否上线"] = df["分数"] >= 660 # 新增一列
print(df.groupby("城市")["分数"].mean()) # 分组统计

df.to_csv("out.csv", index=False, encoding="utf-8-sig")
df2 = pd.read_csv("out.csv")
df3 = pd.read_excel("data.xlsx", sheet_name="Sheet1")
```

备注

- [√] 导出 CSV 给 Excel 看时用 `encoding="utf-8-sig"`（否则中文乱码）
- [!] 链式赋值 `df[df.a>0]["b"]=1` 会警告，用 `.loc[条件, "列"] = 值`
- [!] 处理大文件用 `chunksize=` 分块读
- [√] 读 Excel 需要 `openpyxl` 支持：`pip install openpyxl`

matplotlib —— 画图

定位：最经典的绘图库，论文/报告必备。

用途：把数据画成折线 / 柱状 / 散点 / 直方图，导出 PNG / PDF 放进报告或论文

安装：`pip install matplotlib`

```
import matplotlib.pyplot as plt

x = [1, 2, 3, 4, 5]
y = [1, 4, 9, 16, 25]

plt.rcParams["font.sans-serif"] = ["Microsoft YaHei"] # 让中文不乱码
plt.rcParams["axes.unicode_minus"] = False

plt.plot(x, y, marker="o", label="平方曲线")
plt.bar(x, y, alpha=0.3, label="柱状")
plt.title("示例图")
plt.xlabel("x"); plt.ylabel("y")
plt.legend(); plt.grid(True)
plt.savefig("plot.png", dpi=150, bbox_inches="tight")
plt.show()
```

备注

- [!] **中文显示成方框**：必须设置 `font.sans-serif`（Windows 用 `Microsoft YaHei` / `SimHei`）
- [√] 保存图片要在 `plt.show()` **之前**（show 之后画布会清空）
- [!] 没有图形界面（服务器）时用 `matplotlib.use("Agg")`

其他数据类库（了解即可）

库	定位	安装	什么场景选它
scipy	科学计算（积分、优化、信号、统计）	`pip install scipy`	需要 `scipy.optimize` / `signal` / `stats`

库	定位	安装	什么场景选它
sympy	**符号**数学（解方程、求导、化简）	pip install sympy	要公式推导而不是数值
polars	新一代 DataFrame，比 pandas 快	pip install polars	数据量大（百万行以上）
pyarrow	列式存储 / 格式转换	pip install pyarrow	parquet 文件、加速 pandas
statsmodels	统计建模（回归、检验）	pip install statsmodels	学术统计而不是预测
seaborn	基于 matplotlib 的漂亮统计图	pip install seaborn	想少写代码出好看的图

```
# sympy 符号计算示例
import sympy as sp
x = sp.symbols("x")
print(sp.solve(x**2 - 4, x)) # [-2, 2]
print(sp.diff(sp.sin(x), x)) # cos(x)
print(sp.integrate(x**2, (x, 0, 1))) # 1/3
```

```
# seaborn 一行出图
import seaborn as sns, matplotlib.pyplot as plt
sns.set_theme() # 一键换好看的主题
sns.barplot(x=["A","B"], y=[1,2]); plt.show()
```

三、算法与数据结构

3.1 标准库里的"神器"（不用装，直接 import）

模块	类/函数	干什么	复杂度
collections	deque	**双端队列**，两端 O(1) 增删	比 list 做队列快得多
collections	Counter	计数（本作业挑战 3 的升级版）	O(n)
collections	defaultdict	缺键自动给默认值	省掉 if k in d
collections	OrderedDict	有序字典（3.7+ 普通 dict 也有序了）	—
heapq	heappush/heappop	**最小堆 / 优先队列**	O(log n)
bisect	bisect_left/insort	**二分查找 / 有序插入**	O(log n)
itertools	product/permutations/combinations	全排列、组合、笛卡尔积	竞赛必备
functools	lru_cache	**记忆化**（缓存函数结果）	递归优化神器
functools	reduce	累积计算	—
math	gcd/isqrt/comb/perm/factorial	数论常用	—
random	randint/choice/shuffle/sample	随机	—
copy	deepcopy	深拷贝	—
sys	setrecursionlimit	调大递归深度	深递归必用

```
from collections import Counter, defaultdict, deque
from heapq import heappush, heappop
from bisect import bisect_left, insort
from itertools import permutations, combinations, product
from functools import lru_cache
import math

# ---- 计数 ----
print(Counter("abracadabra").most_common(2)) # [('a', 5), ('b', 2)]

# ---- 缺键自动初始化 ----
graph = defaultdict(list)
graph["A"].append("B") # 不用先判断 A 在不在

# ---- 队列 ----
q = deque([1, 2, 3]); q.appendleft(0); print(q.popleft()) # 0

# ---- 优先队列（最小堆） ----
h = []; heappush(h, 5); heappush(h, 1); print(heappop(h)) # 1
# 最大堆技巧：存负数

# ---- 二分 ----
arr = [1, 3, 5, 7]
print(bisect_left(arr, 5)) # 2
insort(arr, 4); print(arr) # [1,3,4,5,7]

# ---- 组合 / 排列 ----
print(list(combinations([1,2,3], 2))) # [(1,2),(1,3),(2,3)]
print(list(permutations([1,2,3], 2))[:2]) # [(1,2),(1,3)]

# ---- 记忆化递归（斐波那契） ----
@lru_cache(maxsize=None)
def fib(n): return n if n < 2 else fib(n-1) + fib(n-2)
print(fib(100)) # 秒出
```

```
print(math.gcd(12, 18), math.isqrt(17), math.comb(5, 2))
```

> ※ **算法题的心法**：先想清楚"**数据规模**"决定复杂度。
> n ≤ 20 可以指数级暴搜；n ≤ 5000 用 O(n²)；n ≤ 10^5~10^6 必须 O(n log n) 或 O(n)。
> 本作业里的"查重"用 set 就是 O(n)，如果用双重循环就是 O(n²)——这就是选对容器的价值。

3.2 第三方算法库

库	定位	安装
sortedcontainers	有序列表/字典/集合（SortedList **增删查都是 O(log n)**）	pip install sortedcontainers
more-itertools	itertools 的加强版（chunked、windowed、unique_everseen…）	pip install more-itertools
networkx	图论：最短路径、连通分量、拓扑排序	pip install networkx
numpy / scipy	数值算法（矩阵、优化、插值、图算法）	见上一节
python-constraint	约束求解（数独、排班）	pip install python-constraint
pulp / ortools	线性规划 / 运筹优化	pip install pulp

```
# ---- networkx：图算法 ----
import networkx as nx
G = nx.Graph()
G.add_weighted_edges_from([("A","B",1), ("B","C",2), ("A","C",10)])
print(nx.shortest_path(G, "A", "C", weight="weight")) # ['A', 'B', 'C']
print(list(nx.connected_components(G)))
print(list(nx.topological_sort(nx.DiGraph([("A","B"),("B","C")]))))
```

```
# ---- more-itertools：滑动窗口 / 分块 ----
import more_itertools as mit
print(list(mit.windowed([1,2,3,4], 2))) # [(1,2),(2,3),(3,4)]
print(list(mit.chunked([1,2,3,4,5], 2))) # [[1,2],[3,4],[5]]
```

```
# ---- sortedcontainers：需要"一直保持有序"时用它 ----
from sortedcontainers import SortedList
sl = SortedList([5, 1, 3])
sl.add(2); print(sl) # SortedList([1,2,3,5])
print(sl.bisect_left(3)) # 2
```

四、爬虫与网络采集

> [!] **先看合规**：只爬**公开且允许爬取**的数据；遵守目标站点的 robots.txt 与服务条款；
> 控制频率（别把人家站点打挂）；不爬个人隐私、不绕过付费墙/登录墙、不破解验证码。
> 学习和练习请用专门的教学站点（如 books.toscrape.com、httpbin.org、quotes.toscrape.com）。

requests —— 发 HTTP 请求（最常用）

定位：人类友好的 HTTP 客户端，调接口、抓静态页面都用它。

用途：调 REST API、抓网页、下载文件、提交表单 / 登录、上传附件 —— **只要和 HTTP 打交道就先用它**

安装：pip install requests

```
import requests

# ---- GET ----
r = requests.get("https://httpbin.org/get",
 params={"q": "python", "page": 1},
 headers={"User-Agent": "Mozilla/5.0"},
 timeout=10)
print(r.status_code) # 200
print(r.url) # 自动拼好的完整 URL
print(r.text[:200]) # 文本
# print(r.json()) # 如果是 JSON 接口，直接转字典

# ---- POST ----
r = requests.post("https://httpbin.org/post",
 json={"name": "张三"},
 headers={"Authorization": "Bearer xxx"},
 timeout=10)

# ---- 下载文件（流式，适合大文件）----
with requests.get("https://httpbin.org/image/png", stream=True, timeout=10) as r:
 r.raise_for_status()
 with open("img.png", "wb") as f:
 for chunk in r.iter_content(8192):
 f.write(chunk)

# ---- 会话：自动保持 Cookie / 复用连接 ----
s = requests.Session()
s.headers.update({"User-Agent": "Mozilla/5.0"})
s.get("https://httpbin.org/cookies/set?a=1", timeout=10)
print(s.get("https://httpbin.org/cookies", timeout=10).json())
```

备注

- [√] **永远加** timeout=，否则可能永久卡住
- [√] 用 r.raise_for_status() 把 4xx/5xx 变成异常

- [!] 中文乱码：r.encoding = r.apparent_encoding 或直接用 r.content.decode("utf-8")
- [!] r.text 是解码后的文本，r.content 是原始字节
- [√] 批量请求一定要 time.sleep() 限速；并发用 concurrent.futures
- [!] 网上抄的 headers 别带 Content-Length / Host，会出错

httpx —— 支持 async 的现代 HTTP 客户端

定位：requests 的升级版，**原生支持 async/await**，也支持 HTTP/2。

用途：需要**同时发成百上千个请求**时用它（async 并发比多线程省资源），也支持 HTTP/2

安装：pip install httpx

```
import httpx, asyncio

print(httpx.get("https://httpbin.org/get", timeout=10).status_code)

async def main():
 async with httpx.AsyncClient(timeout=10) as client:
 tasks = [client.get("https://httpbin.org/delay/1") for _ in range(5)]
 res = await asyncio.gather(*tasks) # 5 个请求几乎同时完成
 print([r.status_code for r in res])

asyncio.run(main())
```

备注：需要"同时发几百个请求"时，async 版比多线程更省资源。

beautifulsoup4 + lxml —— 解析 HTML

定位：把 HTML 变成可"按标签找"的对象。**bs4 负责好用，lxml 负责快。**

用途：从网页 HTML 里按标签 / class / 属性提取文字、链接、数据；配合 requests 就是最经典的爬虫组合

安装：pip install beautifulsoup4 lxml

```
import requests
from bs4 import BeautifulSoup

html = requests.get("https://books.toscrape.com/", timeout=10).text
soup = BeautifulSoup(html, "lxml") # 用 lxml 解析（比 html.parser 快）

for book in soup.select("article.product_pod")[:3]: # CSS 选择器
 title = book.select_one("h3 a")["title"]
 price = book.select_one("p.price_color").get_text(strip=True)
 link = book.select_one("h3 a")["href"]
 print(title, price, link)

print(soup.title.string)
print(soup.find("h1").get_text(strip=True))
```

选择器速查

写法	含义
soup.select("div.item")	class 为 item 的 div（返回列表）
soup.select_one("div.item")	只要第一个
soup.select("#main")	id 为 main
soup.select("a[href]")	带 href 属性的 a
soup.select("ul > li")	直接子元素
tag.get_text(strip=True)	取纯文本并去空白

写法	含义
tag["href"]	取属性

备注：lxml 在 Windows 上如果装不上，可以先用内置的 "html.parser"（慢一点但零依赖）。

parsel —— 用 XPath/CSS 提取（Scrapy 同款）

用途：在 Scrapy 里、或需要 XPath 精确定位时做 HTML / XML 数据提取

安装：pip install parsel

```
from parsel import Selector
sel = Selector(text="<div><a href='/x'>标题</a></div>")
print(sel.css("a::attr(href)").get()) # /x
print(sel.xpath("//a/text()").get()) # 标题
```

playwright —— 浏览器自动化（现代首选）

定位：真·浏览器驱动，能执行 JS、点按钮、截图、导出 PDF。**比 selenium 更好用。**

用途：抓 **JS 渲染**的动态页面；也能模拟点击 / 填表 / 滚动 / 截图，顺便还能自动化测试和导出 PDF

安装：pip install playwright 然后 playwright install chromium

```
from playwright.sync_api import sync_playwright

with sync_playwright() as p:
 browser = p.chromium.launch(headless=True) # headless=False 可以看过程
 page = browser.new_page()
 page.goto("https://quotes.toscrape.com/js/", wait_until="networkidle")

 # 等元素出现再取（对 JS 渲染的页面很关键）
 page.wait_for_selector(".quote")
 quotes = page.locator(".quote .text").all_inner_texts()
 print(quotes[:3])

 page.fill("input[name=q]", "python") # 填表单
 page.click("button[type=submit]") # 点击
 page.screenshot(path="shot.png", full_page=True)
 browser.close()
```

备注

- [√] 先 wait_for_selector / expect 等元素，别急着取
- [!] 首次要跑 playwright install chromium 下载浏览器（约 100+ MB）
- [√] 异步版：from playwright.async_api import async_playwright
- [!] 别拿它做刷单、抢票等违反网站规则的事

selenium —— 老牌浏览器自动化

用途：用途和 playwright 相同（驱动真实浏览器），生态更老、资料更多；维护老项目时会遇到它

安装：pip install selenium（新版自带驱动管理）

```
from selenium import webdriver
from selenium.webdriver.common.by import By

driver = webdriver.Chrome() # 需要 Chrome
driver.get("https://quotes.toscrape.com/js/")
driver.implicitly_wait(5)
print([e.text for e in driver.find_elements(By.CSS_SELECTOR, ".text")][:3])
driver.quit()
```

备注：生态老、资料多；新项目更推荐 playwright。

scrapy —— 大型爬虫框架

定位：工程化爬虫：自动调度、去重、并发、pipelines、断点续爬。

用途：抓**几万到几十万页**时用：自带调度器、去重、并发、重试、中间件、数据管道和断点续爬

安装：pip install scrapy

```
scrapy startproject myspider
cd myspider
scrapy genspider books books.toscrape.com
scrapy crawl books -o books.json
```

```
# myspider/spiders/books.py
import scrapy

class BooksSpider(scrapy.Spider):
 name = "books"
 start_urls = ["https://books.toscrape.com/"]

 def parse(self, response):
 for book in response.css("article.product_pod"):
 yield {
 "title": book.css("h3 a::attr(title)").get(),
 "price": book.css("p.price_color::text").get(),
 }
 nxt = response.css("li.next a::attr(href)").get()
 if nxt:
 yield response.follow(nxt, self.parse) # 翻页
```

备注：Scrapy 里**不要**自己写 requests；它有自己的下载器和中间件体系。

其他采集类小库

库	干什么	安装
fake-useragent	随机生成 User-Agent	pip install fake-useragent
tqdm	给循环加进度条	pip install tqdm
tenacity	自动重试（网络抖动必备）	pip install tenacity
pyquery	用 jQuery 语法解析 HTML	pip install pyquery
robobrowser	简化表单提交	pip install robobrowser

```
from tqdm import tqdm
from tenacity import retry, stop_after_attempt, wait_fixed
import time

for i in tqdm(range(100)):
 time.sleep(0.01) # 一行就有进度条

@retry(stop=stop_after_attempt(3), wait=wait_fixed(2))
def fetch():
 import requests
 return requests.get("https://httpbin.org/get", timeout=5).status_code

print(fetch()) # 失败自动重试 3 次
```

爬虫工程 Checklist

■ 看 robots.txt、看服务条款、看是否有官方 API（**有 API 就别爬**）

- 设置 User-Agent（表明身份，别伪装成恶意流量）
- timeout + 重试 + 限速（time.sleep 或 tenacity）
- 只取需要的字段，不整站 dump
- 数据落盘（CSV/SQLite/JSON），别只 print
- 出错有日志，中断能续跑

五、网络与安全攻防

> ### [!] 合规声明（请务必读完再往下看）
> 这一节的库都属于"**双刃剑**"。合法使用的前提是**其中至少一条成立**：
> 1. **你自己的设备 / 你自己的网络**（比如扫描 127.0.0.1、自己的云主机）
> 2. **你拿到了书面授权**（公司内部渗透测试、SRC 授权测试、靶场、CTF 比赛）
> 3. **纯学习，且只在自己搭的靶场里练**（推荐 DVWA、OWASP Juice Shop、VulnHub、Hack The Box、TryHackMe、PicoCTF）
> **不要**扫描/攻击未授权的任何主机或网络 —— 在中国可能触犯《刑法》285/286 条、《网络安全法》，在境外同样是重罪。**"我只是试试" 不是理由。**
> 下面所有示例都写成了"只打本机 / 只在自己程序内部"的形式。

5.1 网络基础：socket（标准库）

定位：最底层的网络编程接口，TCP/UDP 都靠它。理解它是理解一切网络库的前提。

```
import socket

# ---- 端口连通性检查（只查自己的服务）----
def check(host, port, timeout=2):
 with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
 s.settimeout(timeout)
 return s.connect_ex((host, port)) == 0

print(check("127.0.0.1", 80))

# ---- 一个只监听本机的迷你 TCP 服务端 ----
# server.py
import socket
srv = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
srv.bind(("127.0.0.1", 9999)); srv.listen(5)
print("listening on 127.0.0.1:9999")
while True:
 conn, addr = srv.accept()
 with conn:
 print("来自", addr, ":", conn.recv(1024).decode(errors="ignore"))
 conn.sendall(b"hello from server")
```

备注：socket 是阻塞的；要高并发用 asyncio、selectors 或第三方框架。

5.2 网络扫描与探测

库	定位	安装
python-nmap	调用 nmap 做端口/服务探测	pip install python-nmap（还要装 nmap 本体）
scapy	**构造/解析/嗅探任意网络包**	pip install scapy
dnspython	DNS 查询与解析	pip install dnspython
whois	域名注册信息	pip install python-whois
pyshark	Python 包 tshark，抓包分析	pip install pyshark

库	定位	安装
netifaces / psutil	看本机网卡 / 网络连接	pip install psutil

```
# ---- python-nmap：只扫本机（学习用法）----
import nmap
nm = nmap.PortScanner()
nm.scan("127.0.0.1", "22-443")
for host in nm.all_hosts():
 print(host, nm[host].state())
 for proto in nm[host].all_protocols():
 for port, info in nm[host][proto].items():
 print(" ", port, info["state"], info.get("name"))
```

```
# ---- scapy：看本机网卡信息 / 解析一个包（只读，无攻击性）----
from scapy.all import get_if_list, IP, TCP, Ether
print(get_if_list()) # 本机网卡列表

pkt = Ether()/IP(dst="127.0.0.1")/TCP(dport=80) # 构造一个包对象
print(pkt.summary()) # 看结构
print(pkt.show(dump=True)[:400])
# 抓包（需要管理员权限；只抓本机流量）
# pkts = sniff(count=5, filter="tcp port 80")
```

```
# ---- dnspython：正常 DNS 查询 ----
import dns.resolver
for rtype in ("A", "MX", "TXT"):
 try:
 print(rtype, [str(r) for r in dns.resolver.resolve("example.com", rtype)])
 except Exception as e:
 print(rtype, "查询失败:", e)
```

备注：scapy 抓包/发原始包在 Windows 上需要 **管理员权限 + Npcap**；Linux 上要 sudo。

5.3 加密与哈希（防御与开发两头都用）

库	定位	安装
hashlib（标准库）	MD5/SHA1/SHA256 摘要	内置
hmac（标准库）	带密钥的摘要（防篡改）	内置
secrets（标准库）	**生成安全随机数 / Token**	内置
cryptography	现代对称/非对称加密、证书	pip install cryptography
pycryptodome	AES/RSA 等，接口更传统	pip install pycryptodome
bcrypt / argon2-cffi	**密码哈希（存数据库专用）**	pip install bcrypt argon2-cffi
pyjwt	JWT 签发与校验	pip install pyjwt

```
import hashlib, hmac, secrets

# ---- 摘要：文件校验 ----
print(hashlib.sha256(b"hello").hexdigest())
def file_sha256(path, chunk=8192):
 h = hashlib.sha256()
 with open(path, "rb") as f:
 while (b := f.read(chunk)):
 h.update(b)
 return h.hexdigest()

# ---- HMAC：验证消息没被改过 ----
key = b"my-secret"
sig = hmac.new(key, b"important data", hashlib.sha256).hexdigest()
print(hmac.compare_digest(sig, hmac.new(key, b"important data", hashlib.sha256).hexdigest()))

# ---- 安全随机 ----
print(secrets.token_urlsafe(24)) # 生成登录 Token / 邀请码
print(secrets.randbelow(100))
```

[!] 重要常识

- **别用 MD5/SHA1 存密码**（可被撞库/彩虹表）。存密码用 bcrypt / argon2，它们自带"慢"和"加盐"
- **别用** random **生成密码/Token**，要用 secrets（random 是可预测的）
- 哈希 ≠ 加密：哈希**不可逆**，加密**可解密**，用途完全不同

```
# ---- cryptography：AES 对称加密 + RSA 非对称 ----
from cryptography.fernet import Fernet
key = Fernet.generate_key() # 保存好这把钥匙！
fernet = Fernet(key)
token = fernet.encrypt(b"secret message")
print(fernet.decrypt(token)) # b'secret message'

from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives import hashes
priv = rsa.generate_private_key(public_exponent=65537, key_size=2048)
pub = priv.public_key()
ct = pub.encrypt(b"hi", padding.OAEP(mgf=padding.MGF1(hashes.SHA256()),
 algorithm=hashes.SHA256(), label=None))
print(priv.decrypt(ct, padding.OAEP(mgf=padding.MGF1(hashes.SHA256()),
 algorithm=hashes.SHA256(), label=None)))
```

```
# ---- bcrypt：正确存密码的方式 ----
import bcrypt
hashed = bcrypt.hashpw(b"user-password", bcrypt.gensalt()) # 存这个到数据库
print(bcrypt.checkpw(b"user-password", hashed)) # True
print(bcrypt.checkpw(b"wrong", hashed)) # False
```

5.4 Web 安全测试（授权环境下）

库	定位	安装
requests / httpx	手工测试接口（改参数看响应）	见爬虫节
mitmproxy	**中间人代理**：抓改手机/浏览器流量，调试和测试神器	pip install mitmproxy
Faker	生成测试数据（别用真实数据）	pip install faker
jwt / pyjwt	分析 JWT（看算法、过期、篡改检测）	pip install pyjwt
bandit	**静态扫描自己代码里的安全漏洞**	pip install bandit
safety / pip-audit	扫描依赖库的已知 CVE	pip install pip-audit

库	定位	安装
OWASP ZAP	专业 Web 扫描（有 Python API）	官网下载

```
# ---- pyjwt：解析（不校验）与校验 ----
import jwt, time
payload = {"sub": "user1", "exp": time.time() + 3600}
token = jwt.encode(payload, "secret", algorithm="HS256")
print(token)
print(jwt.decode(token, "secret", algorithms=["HS256"])) # 校验签名
# [√] 关键：校验时**必须指定 algorithms 列表**，否则可能被算法混淆攻击绕过

# ---- Faker：造测试数据 ----
from faker import Faker
fk = Faker("zh_CN")
print(fk.name(), fk.address(), fk.phone_number(), fk.email())
```

```
# ---- bandit：扫自己代码的安全问题（CI 里常用）----
bandit -r ./myproject
pip-audit # 检查依赖有没有已知漏洞
```

5.5 二进制 / CTF / 逆向（仅限比赛与自有样本）

库	定位	安装
pwntools	CTF 神器：打包/解包、连接、ROP	pip install pwntools（**仅 Linux/macOS**）
capstone	反汇编引擎	pip install capstone
unicorn	CPU 模拟执行	pip install unicorn
pyelftools	解析 ELF 文件	pip install pyelftools
angr	符号执行（自动找路径）	pip install angr
z3-solver	**约束求解**（解方程/逆向约束）	pip install z3-solver
pefile	解析 Windows PE 文件	pip install pefile

```
# ---- pwntools 常用片段（CTF 里连远程题目）----
# from pwn import *
# p = remote("challenge.ctf.example", 1337)
# p.recvuntil(b"name:")
# p.sendline(b"admin")
# print(p.recvline())
# payload = b"A"*40 + p64(0xdeadbeef) # p64/p32 打包整数
```

```
# ---- z3：解约束（数独/逆向题常考）----
from z3 import Ints, Solver, sat
x, y = Ints("x y")
s = Solver()
s.add(x + y == 10, x - y == 4)
print(s.check(), s.model()) # sat [x = 7, y = 3]
```

备注：pwntools 在 Windows 上装不上（要 Linux/WSL），这是它唯一的大门槛。

5.6 防守方视角的库（更推荐往这个方向深耕）

库	干什么	安装
yara-python	**恶意样本特征匹配**（写规则查文件）	pip install yara-python
pyelftools / pefile	分析可执行文件结构	见上

库	干什么	安装
ssdeep / py-tlsh	模糊哈希（找相似样本）	pip install ssdeep
scapy	流量分析与异常检测	见上
geoip2	IP → 地理位置（做日志分析）	pip install geoip2
falconpy（CrowdStrike） / pytenable	商用安全平台 API	pip install falconpy
elasticsearch	日志检索（SIEM 常用）	pip install elasticsearch

```
# ---- yara：写一条规则，扫描目录里的文件 ----
import yara
rule = yara.compile(source=r'''
rule SuspiciousDownloader {
 strings:
 $a = "powershell" nocase
 $b = "DownloadString" nocase
 condition:
 all of them
}
''')
for path in ["sample.exe"]: # 只在你有权分析的文件上跑
 try:
 print(path, rule.match(path))
 except FileNotFoundError:
 pass
```

5.7 安全方向的正确学习路线

1. **基础**：计算机网络（TCP/IP）、Linux 命令、HTTP 协议 —— 这些比工具重要 10 倍
2. **编程**：Python（本手册） + 一门系统语言（C/Go）
3. **靶场**：PortSwigger Web Security Academy（免费）、OWASP Juice Shop、Hack The Box
4. **体系**：OWASP Top 10、CWE Top 25
5. **方向选择**：Web 安全 / 渗透测试 / 二进制逆向 / 流量分析 / 安全开发 / 应急响应
6. **底线**：只打靶场、只测授权目标、漏洞先报告不公开利用细节

六、机器学习与 AI

scikit-learn —— 传统机器学习首选

定位：分类、回归、聚类、降维、模型评估，API 高度统一。

用途：分类、回归、聚类、降维、特征工程、模型评估 —— **表格数据做预测的起点**

安装：pip install scikit-learn

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report
from sklearn.preprocessing import StandardScaler

X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

scaler = StandardScaler() # 特征标准化
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test) # 注意：测试集用 transform，不 fit

model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)
pred = model.predict(X_test)

print("准确率:", accuracy_score(y_test, pred))
print(classification_report(y_test, pred))
print(model.predict_proba(X_test[:1])) # 各类概率
```

备注

- [√] **统一套路**：`fit()` 训练 → `predict()` 预测 → `score()` 评估，换模型只改一行
- [!] 数据泄漏：`fit_transform` 只能用在训练集，测试集必须用 `transform`
- [√] 切分数据要 `random_state` 固定，否则结果每次都不一样
- [√] 类别不均衡看 **f1 / 召回率**，别只看准确率

梯度提升树：XGBoost / LightGBM / CatBoost

定位：**表格数据比赛的常胜将军**，比随机森林更强。

安装：`pip install xgboost lightgbm`

```
import xgboost as xgb
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.metrics import roc_auc_score

X, y = load_breast_cancer(return_X_y=True)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.2, random_state=0)

model = xgb.XGBClassifier(
 n_estimators=300, learning_rate=0.05, max_depth=5,
 subsample=0.8, colsample_bytree=0.8,
 eval_metric="auc", early_stopping_rounds=30, random_state=0,
)
model.fit(Xtr, ytr, eval_set=[(Xte, yte)], verbose=False)
print("AUC:", roc_auc_score(yte, model.predict_proba(Xte)[:, 1]))
```

备注：`early_stopping_rounds` 能在验证集不提升时自动停，省时间还防过拟合。

PyTorch —— 深度学习主流框架

定位：学术界最流行的深度学习框架，动态图、调试友好。

用途：搭建和训练神经网络：图像分类、目标检测、文本模型、**微调大模型**；支持 GPU 加速

安装：`pip install torch torchvision`（官网可按 CUDA 版本生成命令）

```
import torch
import torch.nn as nn
import torch.optim as optim

# ---- 张量 ----
x = torch.randn(3, 4)
print(x.shape, x.device) # torch.Size([3, 4]) cpu
print((x @ x.T).shape)
if torch.cuda.is_available():
 x = x.to("cuda") # 有显卡就搬到 GPU
print(torch.cuda.is_available())

# ---- 一个最小神经网络（手写数字分类）----
from torchvision import datasets, transforms
from torch.utils.data import DataLoader

tf = transforms.ToTensor()
train_ds = datasets.MNIST("./data", train=True, download=True, transform=tf)
train_dl = DataLoader(train_ds, batch_size=64, shuffle=True)

net = nn.Sequential(nn.Flatten(), nn.Linear(28*28, 128), nn.ReLU(), nn.Linear(128, 10))
loss_fn = nn.CrossEntropyLoss()
opt = optim.Adam(net.parameters(), lr=1e-3)

net.train()
for epoch in range(1):
 for xb, yb in train_dl:
 opt.zero_grad() # ① 清梯度
 loss = loss_fn(net(xb), yb) # ② 前向 + 算损失
 loss.backward() # ③ 反向传播
 opt.step() # ④ 更新参数
 print("epoch", epoch, "loss", loss.item())
```

备注

- [√] 训练循环死记四步：`zero_grad → 前向+loss → backward → step`
- [!] 忘了 `zero_grad()`，梯度会累加，模型学不对
- [!] 训练时 `net.train()`，推理时 `net.eval()` + `with torch.no_grad():`
- [√] 有 NVIDIA 显卡装 CUDA 版才快（`pip install torch --index-url https://download.pytorch.org/whl/cu121`）

Transformer / 大模型相关

库	定位	安装
transformers	加载 BERT/GPT 等预训练模型（HuggingFace）	pip install transformers
datasets	一行加载公开数据集	pip install datasets
sentence-transformers	句向量、语义相似度、文本检索	pip install sentence-transformers
openai	调 OpenAI 接口（ChatGPT）	pip install openai
tiktoken	分词 / 估算 token 数（算钱用）	pip install tiktoken
accelerate	多卡/混合精度训练加速	pip install accelerate
peft	参数高效微调（LoRA）	pip install peft
langchain / llama-index	搭 RAG、Agent 编排	pip install langchain

```
# ---- transformers：情感分类（第一次运行会自动下载模型）----
from transformers import pipeline
clf = pipeline("sentiment-analysis", model="distilbert-base-uncased-finetuned-sst-2-english")
print(clf("I love Python!"))
print(clf("This is terrible."))
```

```
# ---- sentence-transformers：算两句话有多像 ----
from sentence_transformers import SentenceTransformer
from sentence_transformers.util import cos_sim
m = SentenceTransformer("all-MiniLM-L6-v2")
v = m.encode(["我爱编程", "我喜歡寫程式", "今天天气不错"])
print(cos_sim(v[0], v[1]).item()) # 接近 1（很像）
print(cos_sim(v[0], v[2]).item()) # 较小（不像）
```

```
# ---- openai：调大模型接口（需要 API Key）----
import os
from openai import OpenAI
client = OpenAI(api_key=os.environ["OPENAI_API_KEY"])
resp = client.chat.completions.create(
 model="gpt-4o-mini",
 messages=[{"role": "user", "content": "用一句话解释什么是元组"}],
)
print(resp.choices[0].message.content)
```

[!] 安全提示：API Key **绝不写进代码**，用环境变量或 `.env`（`python-dotenv`），也别提交到 Git。

中文 NLP 与可视化

库	定位	安装
jieba	中文分词	pip install jieba
wordcloud	词云图	pip install wordcloud
snownlp	中文情感分析（轻量）	pip install snownlp
hanlp / ltp	更专业的中文 NLP 工具包	pip install hanlp
pypinyin	汉字转拼音	pip install pypinyin

```
import jieba
text = "高考志愿填报需要综合考虑分数、城市和专业"
print("/".join(jieba.cut(text)))
print(jieba.lcut_for_search(text))

from pypinyin import lazy_pinyin
print(lazy_pinyin("志愿填报")) # ['zhi', 'yuan', 'tian', 'bao']

from snownlp import SnowNLP
print(SnowNLP("这门课太棒了").sentiments) # 接近 1 = 正面
```

```
# ---- wordcloud：生成词云（需要中文字体）----
from wordcloud import WordCloud
wc = WordCloud(font_path="C:/Windows/Fonts/msyh.ttc", width=800, height=400,
 background_color="white").generate("高分 志愿 专业 城市 未来 努力")
wc.to_file("词云.png")
```

七、Web 后端与接口

Flask —— 最轻量的 Web 框架

定位：几行就能起一个网站/接口，适合小工具和教学。

用途：几行代码起一个网站或接口；适合小工具、教学、给脚本加个网页界面

安装：pip install flask

```
from flask import Flask, request, jsonify, render_template_string

app = Flask(__name__)

@app.get("/")
def index():
 return render_template_string("<h1>你好，{{name}}</h1>", name="世界")

@app.get("/api/hello")
def hello():
 name = request.args.get("name", "匿名")
 return jsonify({"code": 0, "msg": f"你好，{name}"})

@app.post("/api/add")
def add():
 data = request.get_json(force=True)
 return jsonify({"result": data["a"] + data["b"]})

if __name__ == "__main__":
 app.run(host="127.0.0.1", port=5000, debug=True)
```

备注：debug=True 会自动重载、显示详细报错，但**绝不能用在生产环境**（有任意代码执行风险）。

FastAPI —— 现代高性能接口框架（强推）

定位：基于类型注解，**自动生成接口文档**，原生 async，性能接近 Go。

用途：写后端 API：参数自动校验、**自动生成可点击的接口文档**、原生 async；给前端 / App / 大模型做服务

安装：pip install "fastapi[standard]"

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel, Field

app = FastAPI(title="志愿填报 API")

class Student(BaseModel): # 用类型声明请求体，自动校验
 姓名: str = Field(min_length=1, max_length=20)
 高考分数: int = Field(ge=0, le=750)

@app.get("/")
def root():
 return {"msg": "ok"}

@app.post("/analyze")
def analyze(stu: Student):
 if stu.高考分数 >= 700:
 level = "顶尖"
 elif stu.高考分数 >= 660:
 level = "优秀"
 else:
 level = "普通"
 return {"姓名": stu.姓名, "等级": level}

# 运行： uvicorn main:app --reload
# 文档： http://127.0.0.1:8000/docs ← 自动生成，可以直接点着测试
```

```
# 调用自己写的接口
import requests
r = requests.post("http://127.0.0.1:8000/analyze",
 json={"姓名": "张三", "高考分数": 686}, timeout=5)
print(r.json()) # {'姓名': '张三', '等级': '优秀'}
```

备注

- [√] **自动文档**是它最大的优势（/docs Swagger）
- [√] 参数写错类型会被自动拒绝并返回清晰错误
- [!] 启动用 uvicorn main:app --reload，不是 python main.py

其他 Web 相关

库	定位	安装
django	全家桶框架（自带 ORM/后台/用户系统）	pip install django
uvicorn	ASGI 服务器（跑 FastAPI）	pip install uvicorn
gunicorn	WSGI 服务器（Linux 生产用）	pip install gunicorn
pydantic	数据校验与配置（FastAPI 的基石）	pip install pydantic
sqlalchemy	Python 最流行的 ORM	pip install sqlalchemy
starlette	轻量 ASGI 框架（FastAPI 底层）	pip install starlette
jinja2	模板引擎（渲染 HTML）	pip install jinja2
gradio / streamlit	**几行代码做 AI 演示界面**	pip install streamlit

```
django-admin startproject mysite
cd mysite && python manage.py runserver
streamlit run app.py # 秒变网页应用
```

八、办公与自动化

openpyxl —— 读写 Excel(.xlsx)

定位：精确操作单元格、样式、公式。

用途：生成带样式 / 公式 / 图表 / 多 sheet 的 Excel 报表；也用来读取别人给的表格做批量处理

安装：pip install openpyxl

```
from openpyxl import Workbook, load_workbook
from openpyxl.styles import Font, PatternFill, Alignment

wb = Workbook(); ws = wb.active; ws.title = "成绩表"
ws.append(["姓名", "分数", "评价"]) # 写一行

for name, score in [("张三", 686), ("李四", 655)]:
 ws.append([name, score, "优秀" if score >= 660 else "良好"])

ws["A1"].font = Font(bold=True, color="FFFFFF") # 样式
ws["A1"].fill = PatternFill("solid", fgColor="4F81BD")
ws.column_dimensions["A"].width = 14
ws.freeze_panes = "A2" # 冻结首行
ws["C4"] = "=SUM(B2:B3)" # 写公式
wb.save("成绩表.xlsx")

wb2 = load_workbook("成绩表.xlsx")
for row in wb2["成绩表"].iter_rows(min_row=2, values_only=True):
 print(row)
```

备注：.xlsx 用 openpyxl；.xls（老格式）要用 xlrd；只做数据分析直接用 pandas 更省事。

python-docx / python-pptx —— 生成 Word / PPT

用途：自动生成报告、合同、周报（Word）和演示文稿（PPT）—— **把重复排版工作交给代码**

安装：pip install python-docx python-pptx

```
from docx import Document
from docx.shared import Pt

doc = Document()
doc.add_heading("高考志愿分析报告", level=1)
doc.add_paragraph("姓名：张三")
p = doc.add_paragraph("这一行是加粗的")
p.runs[0].bold = True
doc.add_paragraph("第一条建议", style="List Bullet")

t = doc.add_table(rows=1, cols=2)
t.style = "Table Grid"
hdr = t.rows[0].cells; hdr[0].text = "大学"; hdr[1].text = "分数线"
for name, line in [("清华大学", 700), ("浙江大学", 680)]:
 row = t.add_row().cells
 row[0].text = name; row[1].text = str(line)

doc.save("报告.docx")

from pptx import Presentation
from pptx.util import Inches
prs = Presentation()
slide = prs.slides.add_slide(prs.slide_layouts[0])
slide.shapes.title.text = "志愿填报方案"
slide.placeholders[1].text = "张三 • 2026"
prs.save("演示.pptx")
```

PDF 处理

库	定位	安装
pypdf	拆分/合并/旋转/加密 PDF	pip install pypdf
pdfplumber	**提取文字和表格**（最好用）	pip install pdfplumber
PyMuPDF(fitz)	转图片、提取文本，速度极快	pip install pymupdf

库	定位	安装
reportlab	**生成** PDF	pip install reportlab

```
import pdfplumber
with pdfplumber.open("input.pdf") as pdf:
 print("总页数:", len(pdf.pages))
 page = pdf.pages[0]
 print(page.extract_text()[:200])
 for table in page.extract_tables():
 print(table[:2])

# 合并两个 PDF
from pypdf import PdfWriter, PdfReader
w = PdfWriter()
for f in ["a.pdf", "b.pdf"]:
 for page in PdfReader(f).pages:
 w.add_page(page)
w.write("merged.pdf")
```

备注：**扫描件**（图片型 PDF）提不出文字，要配 OCR（见下一节 pytesseract）。

pyautogui —— 桌面自动化（控制鼠标键盘）

定位：模拟人操作鼠标键盘、找图点击。适合重复的桌面流程。

用途：模拟人操作鼠标键盘、找图点击 —— **给没有 API 的桌面软件做自动化**（记得 FAILSAFE 紧急中止）

安装：pip install pyautogui

```
import pyautogui, time

print(pyautogui.size()) # 屏幕分辨率
time.sleep(2) # 给自己留时间切窗口
pyautogui.moveTo(400, 300, duration=0.5)
pyautogui.click()
pyautogui.write("hello world", interval=0.05)
pyautogui.hotkey("ctrl", "s") # 组合键
print(pyautogui.position()) # 当前鼠标位置

# 找图点击（先截一小块图存成 btn.png）
# loc = pyautogui.locateCenterOnScreen("btn.png", confidence=0.9)
# pyautogui.click(loc)
```

备注

- [!] 运行前把鼠标移到屏幕角落可**紧急中止**（pyautogui.FAILSAFE 默认开启）
- [!] 屏幕缩放比例会影响坐标，尽量用"找图"而不是写死坐标
- [√] 更稳定的替代：浏览器用 playwright，Windows 应用可用 pywinauto

定时与调度

库	定位	安装
schedule	最易读的定时任务（进程内）	pip install schedule
APScheduler	功能完整的调度器（cron 表达式）	pip install apscheduler
python-crontab	管理 Linux crontab	pip install python-crontab

```
import schedule, time
def job(): print("该交作业了！")
schedule.every().day.at("20:00").do(job)
schedule.every(10).minutes.do(job)
while True: # 需要程序一直挂着
 schedule.run_pending(); time.sleep(1)
```

备注：Windows 上想"开机自动跑"，最省事是用**任务计划程序**或把脚本放进「启动」文件夹。

九、图像 / 音视频 / OCR

Pillow —— 图像处理基础库

用途：打开 / 裁剪 / 缩放 / 旋转 / 滤镜 / 加文字水印 / 转格式 / 生成缩略图 —— 批量处理图片的第一选择

安装：`pip install pillow`

```
from PIL import Image, ImageDraw, ImageFont, ImageFilter

img = Image.open("input.jpg")
print(img.size, img.mode)
img = img.resize((400, 300))
img = img.rotate(90, expand=True)
img = img.convert("L") # 转灰度
img.filter(ImageFilter.GaussianBlur(2)).save("blur.png")

# 加文字水印
img2 = Image.open("input.jpg").convert("RGB")
d = ImageDraw.Draw(img2)
font = ImageFont.truetype("C:/Windows/Fonts/msyh.ttc", 36)
d.text((20, 20), "小助手", font=font, fill=(255, 0, 0))
img2.save("watermark.jpg")

# 生成缩略图
img.thumbnail((128, 128)); img.save("thumb.png")
```

备注：Pillow 是"单张图简单处理"；要做计算机视觉（识别、检测）用 OpenCV。

OpenCV —— 计算机视觉

用途：图像与视频的进阶处理：边缘检测、人脸识别、目标跟踪、摄像头实时分析

安装：`pip install opencv-python`

```
import cv2

img = cv2.imread("input.jpg") # 注意：BGR 顺序，不是 RGB!
print(img.shape) # (高, 宽, 通道)
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
edges = cv2.Canny(gray, 100, 200) # 边缘检测
cv2.imwrite("edges.png", edges)

# 人脸检测（用 OpenCV 自带模型）
cascade = cv2.CascadeClassifier(
 cv2.data.haarcascades + "haarcascade_frontalface_default.xml")
faces = cascade.detectMultiScale(gray, 1.1, 4)
for (x, y, w, h) in faces:
 cv2.rectangle(img, (x, y), (x+w, y+h), (0, 255, 0), 2)
cv2.imwrite("faces.jpg", img)
print("检测到", len(faces), "张脸")
```

备注

- [!] **OpenCV 默认是 BGR**，matplotlib/PIL 是 RGB，混用颜色会反 → `cv2.cvtColor(img, cv2.COLOR_BGR2RGB)`
- [!] 中文路径读图可能失败，用 `cv2.imdecode(np.fromfile(path, np.uint8), cv2.IMREAD_COLOR)`
- [√] 视频分析也是它：`cap = cv2.VideoCapture("v.mp4")` 循环 `cap.read()`

OCR：文字识别

库	定位	安装
pytesseract	Tesseract 的 Python 封装（需另装 Tesseract 程序）	pip install pytesseract
easyocr	基于深度学习，中文效果更好	pip install easyocr
paddleocr	百度开源，中文识别很强	pip install paddleocr
cnocr	专攻中文 OCR	pip install cnocr

```
# ---- pytesseract ----
import pytesseract
from PIL import Image
pytesseract.pytesseract.tesseract_cmd = r"C:\Program Files\Tesseract-OCR\tesseract.exe"
text = pytesseract.image_to_string(Image.open("shot.png"), lang="chi_sim+eng")
print(text)

# ---- easyocr（首次会下载模型，约 100MB）----
import easyocr
reader = easyocr.Reader(["ch_sim", "en"])
for box, txt, conf in reader.readtext("shot.png"):
 print(round(conf, 2), txt)
```

音视频

库	定位	安装
moviepy	视频剪辑（裁切、拼接、加字幕）	pip install moviepy
ffmpeg-python	用 Python 组装 ffmpeg 命令	pip install ffmpeg-python（需装 ffmpeg）
pydub	音频切片、格式转换、音量	pip install pydub（需 ffmpeg）
mutagen	读写音频元数据（ID3 标签）	pip install mutagen
librosa	音频特征提取（语音/音乐分析）	pip install librosa
whisper	**语音转文字**（OpenAI 开源）	pip install openai-whisper

```
# ---- moviepy：裁剪 + 提取音频 ----
from moviepy import VideoFileClip
clip = VideoFileClip("in.mp4").subclipped(10, 20) # 取 10~20 秒
clip.write_videofile("out.mp4")
clip.audio.write_audiofile("audio.mp3")

# ---- pydub：切音频 ----
from pydub import AudioSegment
song = AudioSegment.from_mp3("song.mp3")
song[:30_000].export("first30s.mp3", format="mp3")
print(len(song) / 1000, "秒")
```

```
# ---- whisper：语音转文字（首次下载模型）----
import whisper
model = whisper.load_model("base")
print(model.transcribe("audio.mp3", language="zh")["text"][:200])
```

二维码 / 条码

```
import qrcode # pip install qrcode[pil]
img = qrcode.make("https://github.com/chenhg5/cc-connect")
img.save("qr.png")

from PIL import Image
from pyzbar.pyzbar import decode # pip install pyzbar
print(decode(Image.open("qr.png"))) # 解码回来
```

十、数据库与缓存

sqlite3 —— 标准库自带的小型数据库

定位：单文件数据库，零安装、零配置，脚本和小项目首选。

用途：一个文件就是一个数据库：**持久化保存爬到的数据**、做本地工具 / 原型的存储（零安装零配置）

安装：内置，`import sqlite3`

```
import sqlite3

conn = sqlite3.connect("school.db")
cur = conn.cursor()
cur.execute("""CREATE TABLE IF NOT EXISTS student(
 id INTEGER PRIMARY KEY AUTOINCREMENT,
 name TEXT NOT NULL, score INTEGER)""")

# [!] 一定要用占位符 ?，不要用 f-string 拼 SQL（会被 SQL 注入）
cur.executemany("INSERT INTO student(name, score) VALUES (?, ?)",
 [("张三", 686), ("李四", 655), ("王五", 700)])
conn.commit()

cur.execute("SELECT name, score FROM student WHERE score >= ? ORDER BY score DESC", (660,))
for row in cur.fetchall():
 print(row)

cur.execute("SELECT AVG(score) FROM student")
print("平均分:", cur.fetchone()[0])

cur.execute("UPDATE student SET score = ? WHERE name = ?", (690, "李四"))
conn.commit()
conn.close()
```

推荐上下文写法：

```
with sqlite3.connect("school.db") as conn: # 自动提交/回滚
 conn.row_factory = sqlite3.Row # 让结果能按列名取
 row = conn.execute("SELECT * FROM student LIMIT 1").fetchone()
 print(dict(row))
```

[!] 数据库安全铁律

- **永远用参数化查询**（? 或 %(name)s），**永远不要** f-string 拼 SQL
- 密码存 **bcrypt 哈希**，不存明文

其他数据库

库	数据库	安装
pymysql	MySQL / MariaDB	pip install pymysql
psycopg[binary]	PostgreSQL	pip install "psycopg[binary]"

库	数据库	安装
pymongo	MongoDB（文档型）	pip install pymongo
redis	Redis（缓存/队列）	pip install redis
sqlalchemy	**ORM**：用 Python 类操作各种数据库	pip install sqlalchemy
peewee / tortoise-orm	更轻量的 ORM	pip install peewee
alembic	数据库迁移（配合 SQLAlchemy）	pip install alembic
duckdb	单机分析型数据库（直接查 CSV/Parquet）	pip install duckdb

```
# ---- SQLAlchemy ORM：用类代替 SQL ----
from sqlalchemy import create_engine, Integer, String, select
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column, Session

class Base(DeclarativeBase): pass

class Student(Base):
 __tablename__ = "student"
 id: Mapped[int] = mapped_column(Integer, primary_key=True)
 name: Mapped[str] = mapped_column(String(20))
 score: Mapped[int] = mapped_column(Integer)

engine = create_engine("sqlite:///school2.db")
Base.metadata.create_all(engine)

with Session(engine) as s:
 s.add_all([Student(name="张三", score=686), Student(name="李四", score=655)])
 s.commit()
 for stu in s.scalars(select(Student).where(Student.score > 660)):
 print(stu.name, stu.score)
```

```
# ---- redis：缓存 / 计数器 / 限流 ----
import redis
r = redis.Redis(host="127.0.0.1", port=6379, decode_responses=True)
r.set("user:1:name", "张三", ex=60) # 60 秒后过期
print(r.get("user:1:name"))
r.incr("page:views") # 访问计数
r.lpush("task_queue", "job1") # 当队列用
```

```
# ---- duckdb：不用装数据库，直接 SQL 查 CSV ----
import duckdb
print(duckdb.sql("SELECT * FROM 'out.csv' LIMIT 3").df())
```

十一、测试、调试与工程化

pytest —— 写测试的标准姿势

用途：给函数写自动化测试，改完代码一键确认「有没有把别的地方改坏」

安装：pip install pytest

用法：文件名 test_*.py，函数名 test_*，然后命令行敲 pytest

```python
# test_score.py
import pytest
from score import analyze # 假设这是你要测试的函数

def test_safe():
 assert analyze(686, 680) == "稳妥"

def test_challenge():
 assert analyze(686, 690) == "冲刺"

@pytest.mark.parametrize("score,line,expect", [
 (700, 700, "稳妥"),
 (695, 700, "冲刺"),
 (680, 700, "暂不推荐"),
])
def test_levels(score, line, expect):
 assert analyze(score, line) == expect

def test_invalid():
 with pytest.raises(ValueError): # 断言"这里应该报错"
 analyze(-1, 600)
```

```
pytest -v # 详细输出
pytest -k safe # 只跑名字含 safe 的
pytest --cov=. --cov-report=term-missing # 覆盖率（需 pytest-cov）
```

代码质量工具链

工具	干什么	安装	用法
ruff	**极快的检查 + 自动格式化**（替代 flake8+isort+black）	pip install ruff	ruff check . --fix
black	代码自动格式化	pip install black	black .
mypy	**静态类型检查**	pip install mypy	mypy main.py
isort	自动整理 import 顺序	pip install isort	isort .
flake8	经典风格检查	pip install flake8	flake8 .
pre-commit	提交前自动跑检查	pip install pre-commit	pre-commit install

备注：个人项目用 ruff 一个就够了（它把好几个工具合了）。**类型注解 + mypy** 能提前抓出"传错类型"的 bug。

日志 / 调试 / 进度

库	定位	安装
loguru	**一行搞定漂亮的日志**（比内置 logging 好用太多）	pip install loguru
icecream	调试打印（自动显示变量名和值）	pip install icecream
pdb / ipdb	断点调试（标准库 pdb 就够了）	内置 / pip install ipdb
tqdm	进度条	pip install tqdm
rich	终端里输出表格/颜色/进度/树	pip install rich
pprint	漂亮的打印嵌套结构	内置

```
from loguru import logger
logger.add("run.log", rotation="10 MB", encoding="utf-8") # 自动切割
logger.info("程序启动")
logger.warning("分数超出范围")
try:
 1 / 0
except ZeroDivisionError:
 logger.exception("出错了") # 自动带完整堆栈

from icecream import ic
x = {"a": 1}
ic(x) # 输出 ic| x: {'a': 1} ← 带变量名
```

```
from rich.console import Console
from rich.table import Table
c = Console()
t = Table(title="志愿表")
t.add_column("大学"); t.add_column("分数线", justify="right")
t.add_row("清华大学", "700"); t.add_row("浙江大学", "680")
c.print(t)
c.print("[bold green]分析完成！[/bold green]")
```

命令行工具

库	定位	安装
argparse	标准库，够用	内置
click	更优雅的命令行框架	pip install click
typer	基于类型注解，**写命令行最快**	pip install typer
rich	好看的终端输出	见上

```
# ---- typer：几行就是一个专业 CLI ----
import typer
app = typer.Typer()

@app.command()
def analyze(score: int, city: str = "杭州", verbose: bool = False):
 """根据分数给出建议"""
 if verbose:
 typer.echo(f"参数: score={score}, city={city}")
 typer.echo("稳妥" if score >= 680 else "冲刺")

if __name__ == "__main__":
 app()
# 运行： python cli.py analyze 686 --city 杭州 --verbose
```

配置与序列化

```
# ---- python-dotenv：把密钥放 .env，别写进代码 ----
import os
from dotenv import load_dotenv # pip install python-dotenv
load_dotenv() # 读取同目录 .env
key = os.getenv("API_KEY")

# ---- PyYAML：读配置文件 ----
import yaml # pip install pyyaml
cfg = yaml.safe_load("""
name: 小助手
score: 686
tags: [计算机, AI]
""")
print(cfg)
print(yaml.safe_dump(cfg, allow_unicode=True))

# ---- orjson：比内置 json 更快的 JSON ----
import orjson # pip install orjson
data = orjson.loads(b'{"a": 1}')
print(orjson.dumps(data))
```

[!] 安全：.env、config.yaml 里可能有密钥，**必须加进 .gitignore**，别提交到 GitHub。

十二、快速选型表 + 学习路线

12.1 按任务查库（一页速查）

任务	首选	备选
HTTP 请求	requests	httpx（异步）
解析 HTML	beautifulsoup4 + lxml	parsel、pyquery
大爬虫	scrapy	自己写 + httpx
JS 渲染页面	playwright	selenium
表格数据	pandas	polars（大数据）
数值计算	numpy	scipy
画图	matplotlib	seaborn、plotly
机器学习	scikit-learn	xgboost、lightgbm
深度学习	pytorch	tensorflow
大模型调用	openai	transformers（本地）
中文分词	jieba	hanlp
Web 接口	fastapi + uvicorn	flask
网站（带后台）	django	flask
Excel	openpyxl / pandas	xlsxwriter
Word / PPT	python-docx / python-pptx	用 win32com 调 Office
PDF 提取	pdfplumber	PyMuPDF
PDF 生成	reportlab	weasyprint（HTML→PDF）
图像处理	Pillow	opencv-python
视频剪辑	moviepy	ffmpeg-python
语音转文字	whisper	云厂商 ASR API
OCR	paddleocr / easyocr	pytesseract

任务	首选	备选
数据库	sqlite3（小） / sqlalchemy（通用）	pymysql、pymongo
缓存	redis	functools.lru_cache（单机）
测试	pytest	unittest
代码质量	ruff	black + flake8
日志	loguru	logging
命令行	typer	click、argparse
桌面自动化	pyautogui	pywinauto
定时任务	schedule / APScheduler	系统自带计划任务
密码哈希	bcrypt / argon2-cffi	[×] 别用 md5
加密	cryptography	pycryptodome
网络安全测试	scapy、python-nmap、mitmproxy	见第五章合规声明

12.2 学习路线（按兴趣选一条）

① 数据分析 / AI 方向

numpy → pandas → matplotlib → scikit-learn → pytorch → transformers

② 爬虫 / 数据采集方向

requests → beautifulsoup4 → pandas 存数据 → playwright（JS 页面）→ scrapy（工程化）→ 反爬与限速

③ Web 开发方向

flask（入门）→ fastapi（接口）→ sqlalchemy（数据库）→ django（大项目）→ 部署（Docker + Nginx）

④ 网络安全方向（务必读第五章合规声明）

计算机网络 + Linux → socket/requests → Burp/ZAP 手工测试 → scapy 流量分析 → yara/capstone 样本分析 → 靶场刷题（HTB / PortSwigger）

⑤ 自动化 / 效率方向

openpyxl、python-docx 处理文档 → playwright 网页自动化 → pyautogui 桌面自动化 → schedule 定时 → 打包成 exe（pyinstaller）

⑥ 通用基本功（所有人都该会）

pytest 写测试 · ruff 保持整洁 · loguru 打日志 · python-dotenv 管密钥 · git 管版本 · venv 隔离环境

12.3 常用安装命令合集（可直接复制）

```
# 数据
pip install numpy pandas matplotlib scipy sympy seaborn polars openpyxl

# 爬虫
pip install requests httpx beautifulsoup4 lxml parsel scrapy playwright tqdm tenacity

# 网络与安全（仅授权/靶场使用）
pip install scapy python-nmap dnspython cryptography pycryptodome bcrypt pyjwt mitmproxy faker pip-audit
     bandit

# AI / 机器学习
pip install scikit-learn xgboost lightgbm transformers datasets sentence-transformers openai jieba wordc
    loud

# Web
pip install "fastapi[standard]" flask django uvicorn sqlalchemy pydantic

# 办公自动化
pip install python-docx python-pptx pdfplumber pypdf reportlab pyautogui schedule

# 图像 / 音视频 / OCR
pip install pillow opencv-python moviepy pydub mutagen qrcode pyzbar

# 数据库
pip install pymysql pymongo redis duckdb peewee

# 工程化
pip install pytest ruff mypy loguru rich typer click python-dotenv pyyaml orjson ipython

# 国内加速（加在命令末尾）
# -i https://pypi.tuna.tsinghua.edu.cn/simple
```

12.4 三条选库心得

1. 能用标准库就别装第三方。

`collections`、`itertools`、`functools`、`sqlite3`、`json`、`pathlib`、`argparse`
已经能解决大量问题，而且零依赖、不会被版本坑。

2. 一个方向只精一个库。

比如爬虫别同时学 requests / httpx / aiohttp / scrapy 四个；先 `requests` 打牢，再按需要扩展。

3. 先读官方文档的 Quickstart（15 分钟），再写自己的例子。

本手册的作用是"**告诉你有什么**"，具体 API 永远以官方文档为准（搜 `库名 + 官方文档` 或去 GitHub README）。

---

*（本文件由 AI 助手整理，供学习参考；示例以"合法、安全、可运行"为前提。第三方库 API
会随版本变化，请以官方文档为准。）*